

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : CYBER SECURITY

AIM : Implement digital signature algorithms such as RSA-based signatures, and verify the integrity and authenticity of digitally signed messages.

CODE CLI:

```
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.exceptions import InvalidSignature

def main():
    print("=" * 60)
    print(" RSA DIGITAL SIGNATURE - INTERACTIVE TERMINAL")
    print("=" * 60)

    print("\n[+] Generating RSA 2048-bit Key Pair...")
    private_key = rsa.generate_private_key(public_exponent=65537,
key_size=2048)
    public_key = private_key.public_key()
    print("[+] Keys generated successfully!\n")

    while True:
        print("-" * 60)
        user_input = input("Enter a message to sign (or type 'exit' to
quit): ")
        if user_input.lower() == 'exit':
            print("Exiting...")
            break

        message_bytes = user_input.encode('utf-8')

        signature = private_key.sign(
            message_bytes,
            padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
salt_length=padding.PSS.MAX_LENGTH),
            hashes.SHA256()
        )
        print(f"\n[SIGNING] Message signed!")
        print(f"Signature (Hex snippet): {signature.hex()[:60]}...")
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : CYBER SECURITY

```
print("\n--- Verification Test 1: Authentic Message ---")
try:
    public_key.verify(
        signature,
        message_bytes,
        padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
salt_length=padding.PSS.MAX_LENGTH),
        hashes.SHA256()
    )
    print("RESULT: SUCCESS (Valid signature, message is authentic
and untampered)")
except InvalidSignature:
    print("RESULT: FAILED")

print("\n--- Verification Test 2: Tampered Message Simulation
---")

tampered_input = user_input + " [TAMPERED]"
tampered_bytes = tampered_input.encode('utf-8')
print(f"Simulating altered message received: '{tampered_input}'")

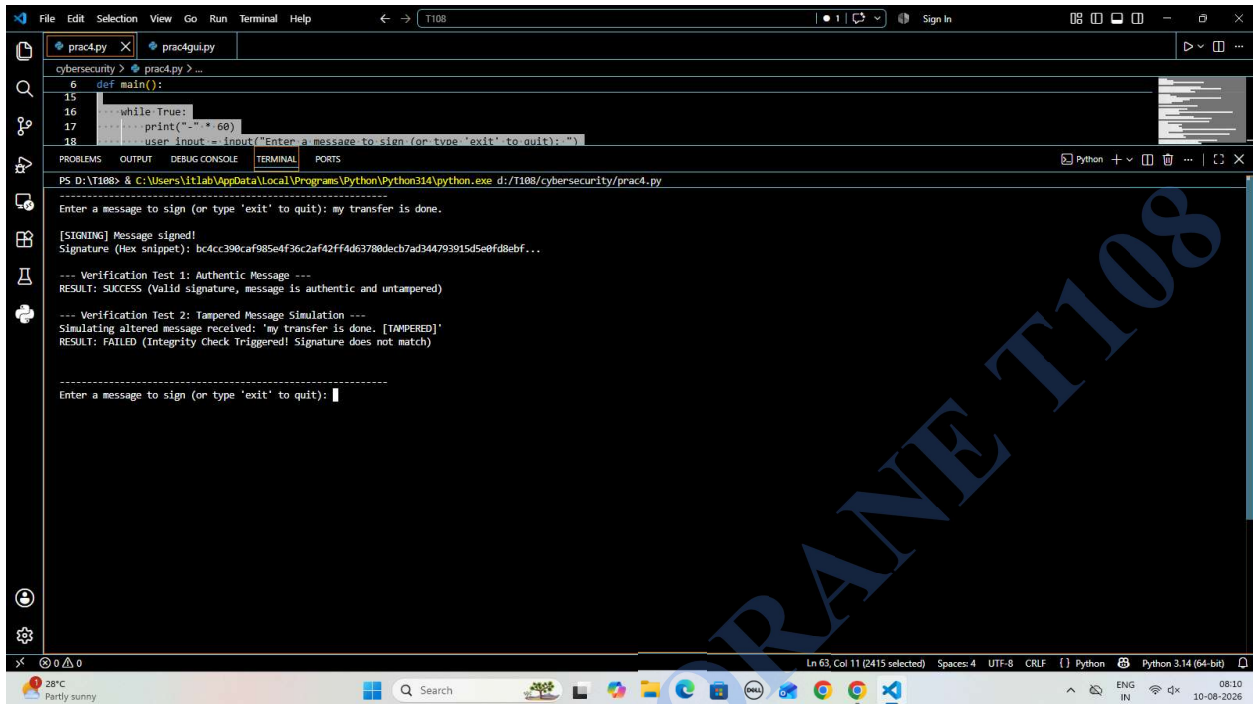
try:
    public_key.verify(
        signature,
        tampered_bytes,
        padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
salt_length=padding.PSS.MAX_LENGTH),
        hashes.SHA256()
    )
    print("RESULT: SUCCESS")
except InvalidSignature:
    print("RESULT: FAILED (Integrity Check Triggered! Signature
does not match)")
print("\n")

if __name__ == "__main__":
    main()
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : CYBER SECURITY

OUTPUT



```
cybersecurity > prac4.py > ...
6 def main():
15
16     while True:
17         print("-" * 60)
18         user_input = input("Enter a message to sign (or type 'exit' to quit): ")

PS D:\T108> & C:\Users\vitlab\AppData\Local\Programs\Python\Python314\python.exe d:/T108/cybersecurity/prac4.py
Enter a message to sign (or type 'exit' to quit): my transfer is done.

[SIGNING] Message signed!
Signature (Hex snippet): bc4cc390caf98e4f36c2af42ff4d63780decb7ad344793915d5e0fd8ebf...

--- Verification Test 1: Authentic Message ---
RESULT: SUCCESS (Valid signature, message is authentic and untampered)

--- Verification Test 2: Tampered Message Simulation ---
Simulating altered message received: 'my transfer is done. [TAMPERED]'
RESULT: FAILED (Integrity Check Triggered! Signature does not match)

Enter a message to sign (or type 'exit' to quit):
```

CODE GUI:

```
import tkinter as tk
from tkinter import messagebox, scrolledtext
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.exceptions import InvalidSignature

class RSASignatureApp:
    def __init__(self, root):
        self.root = root
        self.root.title("RSA Digital Signature Tool")
        self.root.geometry("650x700")
        self.root.config(bg="#f0f0f0")

        self.private_key = None
        self.public_key = None
        self.current_signature = None

        self.create_widgets()
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : CYBER SECURITY

```
def create_widgets(self):
    title_label = tk.Label(self.root, text="RSA Digital Signature &
Verification", font=("Arial", 14, "bold"), bg="#f0f0f0")
    title_label.pack(pady=10)

    frame_keys = tk.LabelFrame(self.root, text=" 1. Key Management ",
font=("Arial", 10, "bold"), bg="#f0f0f0", padx=10, pady=10)
    frame_keys.pack(fill="x", padx=15, pady=5)

    self.btn_gen = tk.Button(frame_keys, text="Generate New RSA Key
Pair", command=self.generate_keys, bg="#4CAF50", fg="white",
font=("Arial", 10, "bold"))
    self.btn_gen.pack(fill="x")

    self.lbl_key_status = tk.Label(frame_keys, text="Status: No keys
generated yet.", fg="red", bg="#f0f0f0")
    self.lbl_key_status.pack(anchor="w", pady=5)

    frame_sign = tk.LabelFrame(self.root, text=" 2. Message Signing ",
font=("Arial", 10, "bold"), bg="#f0f0f0", padx=10, pady=10)
    frame_sign.pack(fill="x", padx=15, pady=5)

    tk.Label(frame_sign, text="Enter Message to Sign:",
bg="#f0f0f0").pack(anchor="w")

    self.txt_message = tk.Text(frame_sign, height=4, width=70)
    self.txt_message.pack(fill="x", pady=5)
    self.txt_message.insert(tk.END, "Confidential: Transfer $10,000 to
Account A.")

    self.btn_sign = tk.Button(frame_sign, text="Sign Message",
command=self.sign_msg, bg="#2196F3", fg="white", font=("Arial", 10,
"bold"), state="disabled")
    self.btn_sign.pack(fill="x", pady=5)

    tk.Label(frame_sign, text="Generated Signature (Hex):",
bg="#f0f0f0").pack(anchor="w")

    self.txt_signature = scrolledtext.ScrolledText(frame_sign,
height=4, width=70)
    self.txt_signature.pack(fill="x", pady=5)
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : CYBER SECURITY

```
frame_verify = tk.LabelFrame(self.root, text=" 3. Verification ",
font=("Arial", 10, "bold"), bg="#f0f0f0", padx=10, pady=10)
frame_verify.pack(fill="x", padx=15, pady=5)

tk.Label(frame_verify, text="Message to Verify (Edit this to test
tampering):", bg="#f0f0f0").pack(anchor="w")
self.txt_verify_msg = tk.Text(frame_verify, height=4, width=70)
self.txt_verify_msg.pack(fill="x", pady=5)
self.txt_verify_msg.insert(tk.END, "Confidential: Transfer $10,000
to Account A.")

self.btn_verify = tk.Button(frame_verify, text="Verify Signature",
command=self.verify_sig, bg="#FF9800", fg="white", font=("Arial", 10,
"bold"), state="disabled")
self.btn_verify.pack(fill="x", pady=5)

self.lbl_result = tk.Label(frame_verify, text="Verification
Result: Not verified", font=("Arial", 11, "bold"), bg="#f0f0f0")
self.lbl_result.pack(anchor="w", pady=5)

def generate_keys(self):
    try:
        self.private_key =
rsa.generate_private_key(public_exponent=65537, key_size=2048)
        self.public_key = self.private_key.public_key()
        self.lbl_key_status.config(text="Status: RSA 2048-bit Key Pair
Active.", fg="green")
        self.btn_sign.config(state="normal")
        self.btn_verify.config(state="normal")
        messagebox.showinfo("Success", "RSA Key Pair successfully
generated!")
    except Exception as e:
        messagebox.showerror("Error", f"Key generation failed:
{str(e)}")

def sign_msg(self):
    message_text = self.txt_message.get("1.0",
tk.END).strip().encode('utf-8')
    if not message_text:
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : CYBER SECURITY

```
messagebox.showwarning("Warning", "Message cannot be empty!")
return

try:
    self.current_signature = self.private_key.sign(
        message_text,
        padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
salt_length=padding.PSS.MAX_LENGTH),
        hashes.SHA256()
    )
    self.txt_signature.delete("1.0", tk.END)
    self.txt_signature.insert(tk.END,
self.current_signature.hex())

    self.txt_verify_msg.delete("1.0", tk.END)
    self.txt_verify_msg.insert(tk.END,
message_text.decode('utf-8'))

    messagebox.showinfo("Success", "Message signed successfully!")
except Exception as e:
    messagebox.showerror("Error", f"Signing failed: {str(e)}")

def verify_msg(self):
    if not self.current_signature:
        messagebox.showwarning("Warning", "Please sign a message
first!")
        return

    verify_text = self.txt_verify_msg.get("1.0",
tk.END).strip().encode('utf-8')

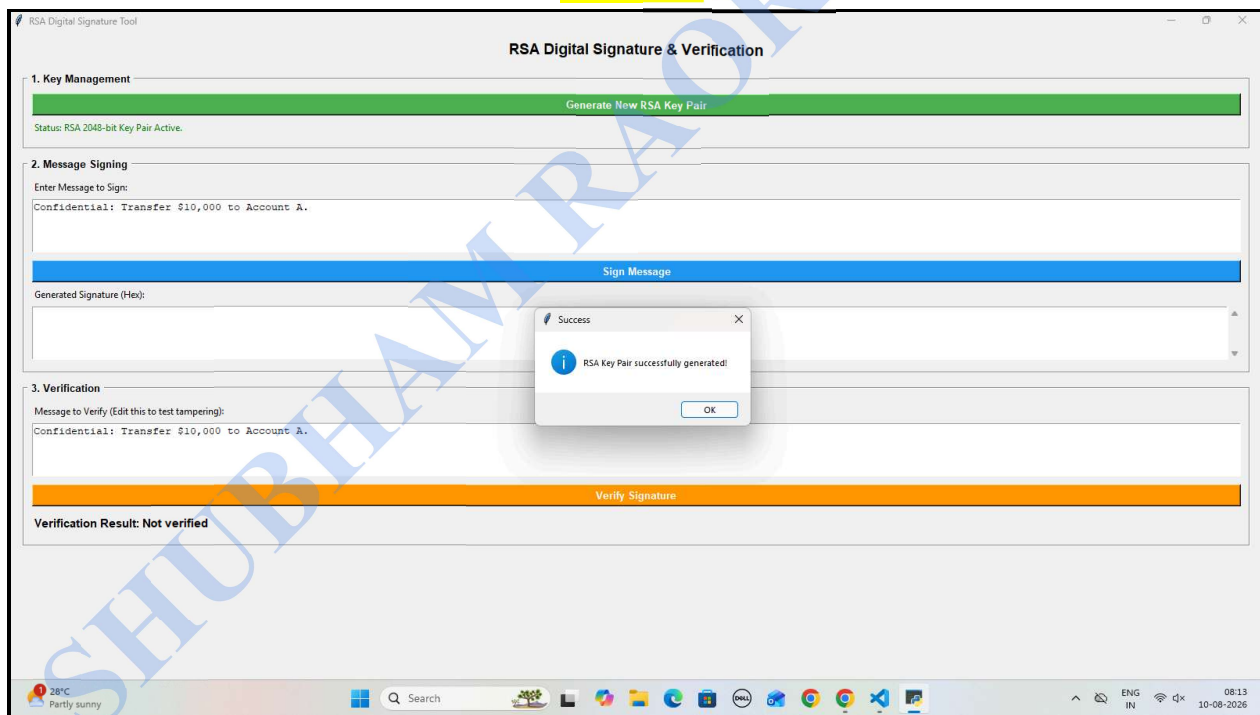
    try:
        self.public_key.verify(
            self.current_signature,
            verify_text,
            padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
salt_length=padding.PSS.MAX_LENGTH),
            hashes.SHA256()
        )
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : CYBER SECURITY

```
self.lbl_result.config(text="Verification Result: SUCCESS  
(Authentic & Intact)", fg="green")  
except InvalidSignature:  
    self.lbl_result.config(text="Verification Result: FAILED  
(Tampered or Invalid)", fg="red")  
except Exception as e:  
    messagebox.showerror("Error", f"Verification error: {str(e)}")  
  
def verify_sig(self):  
    self.verify_msg()  
  
if __name__ == "__main__":  
    root = tk.Tk()  
    app = RSASignatureApp(root)  
    root.mainloop()
```

OUTPUT



SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : CYBER SECURITY

The image displays two screenshots of the 'RSA Digital Signature Tool' interface. The top screenshot shows the 'Message Signing' section where a message 'Confidential: Transfer \$10,000 to Account A.' is signed, resulting in a long hexadecimal signature. A 'Success' dialog box is overlaid on the 'Verification' section, stating 'Message signed successfully!'. The bottom screenshot shows the same tool with the 'Verify Signature' button highlighted, and the 'Verification Result' is 'SUCCESS (Authentic & Intact)'. A large diagonal watermark 'SHUBHAM RAORANE T108' is visible across both screenshots.

RSA Digital Signature Tool

RSA Digital Signature & Verification

1. Key Management

Generate New RSA Key Pair

Status: RSA 2048-bit Key Pair Active.

2. Message Signing

Enter Message to Sign:

Confidential: Transfer \$10,000 to Account A.

Sign Message

Generated Signature (Hex):

9b15337ac571322c1b50ae807757a15094c455fd435101245cbbd1c15fc0001c6fe1131f24165bf4bd3349c3f028339ddfb5bd261e5c70084980ec0510d89996965b086371e2f3bea80adad2130624938ef89dd611f3ab00212d4e45d6500d3d91335129d29e9a4eb0ab7cfa8bee3497639f7357aacd5245b6780d6c7c4ac11c119a103b497dc6dab2e029ca9b9a985029b0ae59239a4ec5ca4fa4ac205305a9d56c453fd118bd2c2c5cabeb73672fc17794bce58900b675944e698905fa630f74f56e4c00ea33a5ca72859c36598b291fb7b54d62aeb59e762d852c168af988d1ff7d4dafc90c9fc96901f37a1bdac641dd1ce6d986b43ec1532a173489

3. Verification

Message to Verify (Edit this to test tampering):

Confidential: Transfer \$10,000 to Account A.

Verify Signature

Verification Result: Not verified

Success

Message signed successfully!

OK

28°C Partly sunny

Search

ENG IN 08:13 10-08-2026

RSA Digital Signature Tool

RSA Digital Signature & Verification

1. Key Management

Generate New RSA Key Pair

Status: RSA 2048-bit Key Pair Active.

2. Message Signing

Enter Message to Sign:

Confidential: Transfer \$10,000 to Account A.

Sign Message

Generated Signature (Hex):

9b15337ac571322c1b50ae807757a15094c455fd435101245cbbd1c15fc0001c6fe1131f24165bf4bd3349c3f028339ddfb5bd261e5c70084980ec0510d89996965b086371e2f3bea80adad2130624938ef89dd611f3ab00212d4e45d6500d3d91335129d29e9a4eb0ab7cfa8bee3497639f7357aacd5245b6780d6c7c4ac11c119a103b497dc6dab2e029ca9b9a985029b0ae59239a4ec5ca4fa4ac205305a9d56c453fd118bd2c2c5cabeb73672fc17794bce58900b675944e698905fa630f74f56e4c00ea33a5ca72859c36598b291fb7b54d62aeb59e762d852c168af988d1ff7d4dafc90c9fc96901f37a1bdac641dd1ce6d986b43ec1532a173489

3. Verification

Message to Verify (Edit this to test tampering):

Confidential: Transfer \$10,000 to Account A.

Verify Signature

Verification Result: SUCCESS (Authentic & Intact)

28°C Partly sunny

Search

ENG IN 08:13 10-08-2026

Function Description

- `rsa.generate_private_key()` – Generates the RSA private key.
- `private_key.public_key()` – Generates the public key from the private key.
- `input()` – Takes the message from the user.
- `private_key.sign()` – Creates a digital signature for the message.
- `hashes.SHA256()` – Creates a secure hash of the message.
- `public_key.verify()` – Verifies whether the message matches the digital signature.
- `try-except` – Handles successful and failed signature verification.

Observation

The program successfully generates an RSA digital signature for the user-entered message. When the same message is entered during verification, the signature is successfully verified. When the message is modified, verification fails. This proves that RSA digital signatures can be used to ensure message integrity and authenticity.

ALGORITHM:

Generate RSA keys – Create a private key and public key.

Enter message – User enters the message.

Hash message – Generate SHA-256 hash of the message.

Sign – Use the RSA private key to create the digital signature.

Verify – Use the RSA public key to verify the signature.

Compare – If verification succeeds, the message is authentic and unchanged; otherwise, it has been modified.

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : CYBER SECURITY

SENDER

|

Enter Message

|

SHA-256

|

RSA Private Key

|

Digital Signature

|

Message + Signature

|

↓

RECEIVER

|

Received Message

|

SHA-256

|

RSA Public Key

|

Verification

/ \

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : CYBER SECURITY

SUCCESS FAILED

| |

Authentic Modified/

& unchanged invalid

SHUBHAM RAORANE T108